

Experiment 08:

Step 1: Import required libraries

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as s
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

Step 2: Load the dataset

Code:

```
iris=load_iris()
print(iris.data[:5])
```

Output:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

Step 3: Convert the Dataset into Dataframes

Code:

```
df=pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
df.head()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Step 4: Other Information-check, Datatypes and Null Values

Code:

```
print("\nShape : ",df.shape)
print("\nInfo : \n",df.info)
print("\nDescriptive Statistics \n",df.describe())
print("\nMissing Values : \n",df.isnull().sum())
```

Output:

```
Shape : (150, 4)
Info :
... <bound method DataFrame.info of      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0          5.1          3.5          1.4          0.2
1          4.9          3.0          1.4          0.2
2          4.7          3.2          1.3          0.2
3          4.6          3.1          1.5          0.2
4          5.0          3.6          1.4          0.2
..          ...          ...          ...          ...
145         6.7          3.0          5.2          2.3
146         6.3          2.5          5.0          1.9
147         6.5          3.0          5.2          2.0
148         6.2          3.4          5.4          2.3
149         5.9          3.0          5.1          1.8

[150 rows x 4 columns]>
Descriptive Statistics
      sepal length (cm)  sepal width (cm)  petal length (cm) \
count          150.000000          150.000000          150.000000
mean             5.843333             3.057333             3.758000
std              0.828066             0.435866             1.765298
min              4.300000             2.000000             1.000000
25%              5.100000             2.800000             1.600000
50%              5.800000             3.000000             4.350000
75%              6.400000             3.300000             5.100000
max              7.900000             4.400000             6.900000

      petal width (cm)
count          150.000000
mean             1.199333
std              0.762238
min              0.100000
25%              0.300000
50%              1.300000
75%              1.800000
max              2.500000

Missing Values :
sepal length (cm)    0
```

```
sepal width (cm)    0
petal length (cm)   0
petal width (cm)    0
dtype: int64
```

Step 5: Select Features and Targets

Code:

```
X=df.values
print(X[:5])
```

Output:

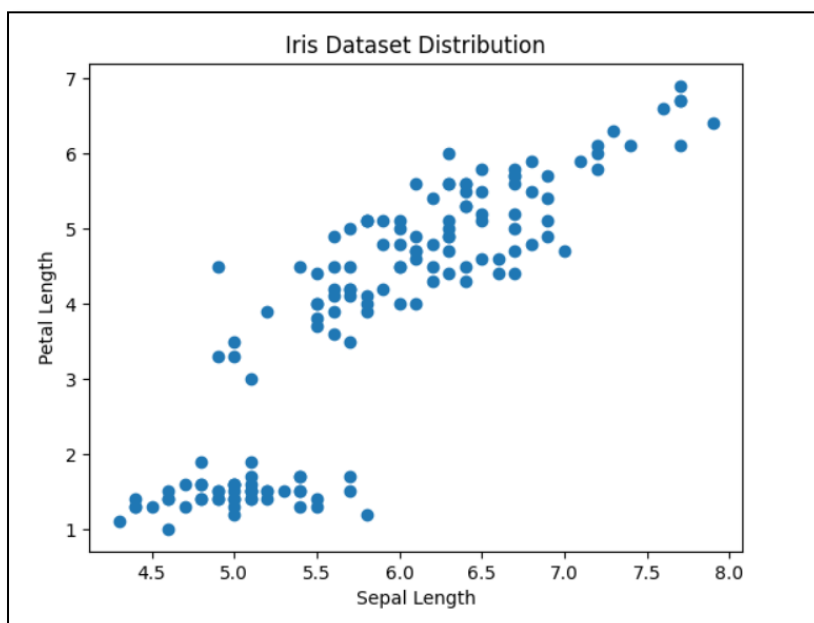
```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

Step 6: Visualization of Dataset

Code:

```
plt.scatter(df['sepal length (cm)'],df['petal length (cm)'])
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.title("Iris Dataset Distribution")
plt.show()
```

Output:



Step 7: Standardization

Code:

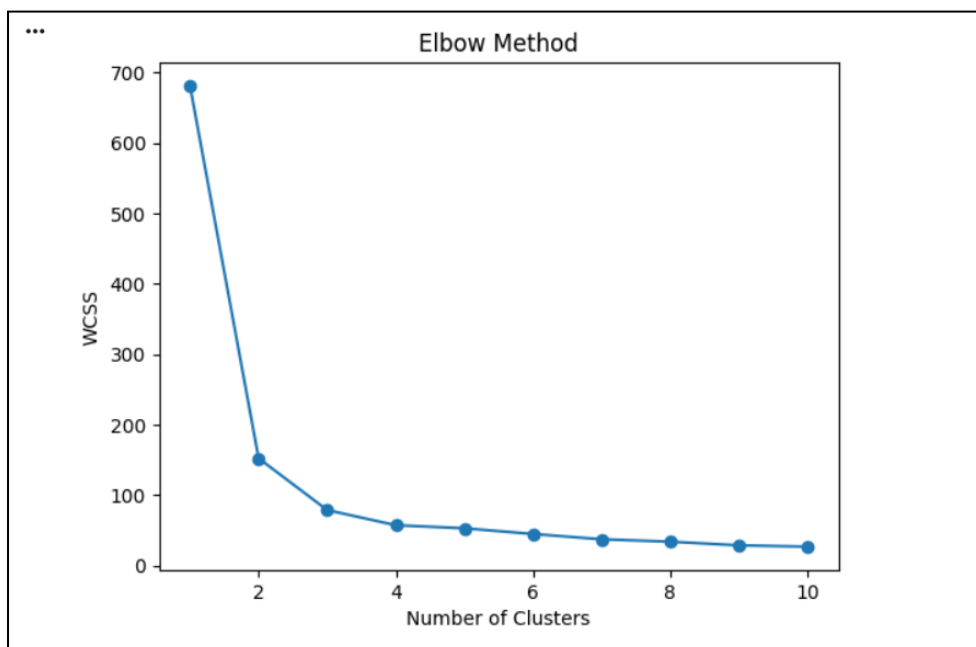
```
scaler=StandardScaler()  
X_scaled=scaler.fit_transform(X)
```

Step 8: Determine the Optimal Number of Clusters using Elbow Method

Code:

```
wcss=[]  
  
for i in range(1,11):  
    kmeans=KMeans(n_clusters=i)  
    kmeans.fit(X)  
    wcss.append(kmeans.inertia_)  
  
plt.plot(range(1,11), wcss, marker='o')  
plt.xlabel("Number of Clusters")  
plt.ylabel("WCSS")  
plt.title("Elbow Method")  
plt.show()
```

Output:



Observation:

- The Elbow Method was used to determine a suitable value of **K** by plotting the **WCSS** against the number of clusters.
- The graph shows a sharp decrease in WCSS up to **K = 3**, followed by a slower decrease.
- Hence, **K = 3** was selected as the suitable number of clusters.

Step 9: Apply KMean Clustering

Code:

```
kmeans= KMeans(n_clusters=3)
kmeans.fit(X_scaled)
```

Output:

```
KMeans
KMeans(n_clusters=3)
```

Observation:

- K-Means clustering was applied with **K = 3** on the standardized dataset.
- The algorithm grouped the 150 Iris observations into three clusters based on similarity in their standardized feature values.

Step 10: Predict Cluster Labels

Code:

```
clusters = kmeans.predict(X_scaled)
df['Clsuter']=clusters
print(df.head())
```

Observation:

- Each observation was assigned a **cluster label** using the trained K-Means model.
- The labels **0, 1 and 2** represent the three clusters and are only identifiers for the groups; they do not represent numerical rankings or species names.

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
	Clsuter				
0	1				
1	1				
2	1				
3	1				
4	1				

Step 11: Find Cluster Centers

Code:

```
centroids=kmeans.cluster_centers_  
print(centroids)
```

Output:

```
[[-0.01139555 -0.87600831  0.37707573  0.31115341]  
 [-1.01457897  0.85326268 -1.30498732 -1.25489349]  
 [ 1.16743407  0.14530299  1.00302557  1.0300019  ]]
```

Observation:

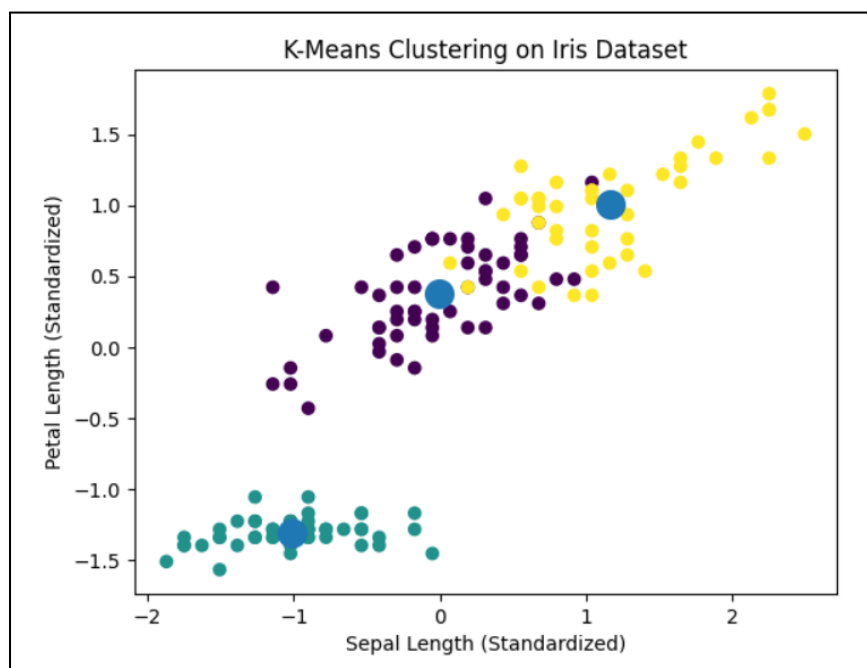
- The **cluster_centers** were obtained from the trained K-Means model.
- Each **center** represents the mean position of the observations assigned to that cluster across the four standardized features.
- These **centroids** represent the central location of the three formed clusters.

Step 12: Visualize Clusters

Code:

```
plt.scatter(X_scaled[:,0], X_scaled[:,2], c=clusters)  
plt.scatter(centroids[:,0],centroids[:,2],marker='o',s=200)  
plt.xlabel("Sepal Length")  
plt.ylabel("Petal Length")  
plt.title("k-Means Clustering on Iris Dataset")  
plt.show()
```

Output:



Observation:

- The clusters were visualized using **Sepal Length (X_scaled[:,0])** on the X-axis and **Petal Length (X_scaled[:,2])** on the Y-axis.
- The different colors represent the cluster labels, while the large circular points represent the corresponding cluster centroids.

Why X_scaled[:,0] and X_scaled[:,2]?

X_scaled[:,0] selects the **first feature, Sepal Length**, and X_scaled[:,2] selects the **third feature, Petal Length**. This allows the four-dimensional clustering result to be displayed in a two-dimensional graph.

Step 13: Display Final Cluster Sizes and Final Centroids**Code:**

```
final_labels = clusters
cluster_sizes = np.bincount(final_labels)
print("Final Cluster Sizes:")
for i, size in enumerate(cluster_sizes):
    print(f"Cluster {i}: {size} observations")
print("\nFinal Centroids:")
print(centroids)
```

Output:

```
Final Cluster Sizes:
Cluster 0: 56 observations
Cluster 1: 50 observations
Cluster 2: 44 observations

Final Centroids:
[[-0.01139555 -0.87600831  0.37707573  0.31115341]
 [-1.01457897  0.85326268 -1.30498732 -1.25489349]
 [ 1.16743407  0.14530299  1.00302557  1.0300019  ]]
```

Observation:

- The final cluster sizes and centroid values were displayed after clustering.
- The three clusters contain **56, 50 and 44 observations**, giving a total of 150 observations.
- The centroids shown are the final centroid positions obtained from the K-Means model.

Why display the centroids again?

Step 11 obtains the centroid values, whereas Step 13 presents them together with the **final cluster sizes** to summarize the completed clustering result.

Step 14: Convert Centroids Back to Original Units

Code:

```
final_centroids = scaler.inverse_transform(centroids)
print("Final Centroids in Original Units:")
print(final_centroids)
```

Output:

```
Final Centroids in Original Units:
[[5.83392857 2.67678571 4.42142857 1.43571429]
 [5.006      3.428      1.462      0.246      ]
 [6.80681818 3.12045455 5.52272727 1.98181818]]
```

Observation:

- The standardized centroids were converted back to the **original measurement units** using `inverse_transform()`.
 - This makes the centroid values easier to interpret in terms of the original Iris measurements such as centimetres.
-

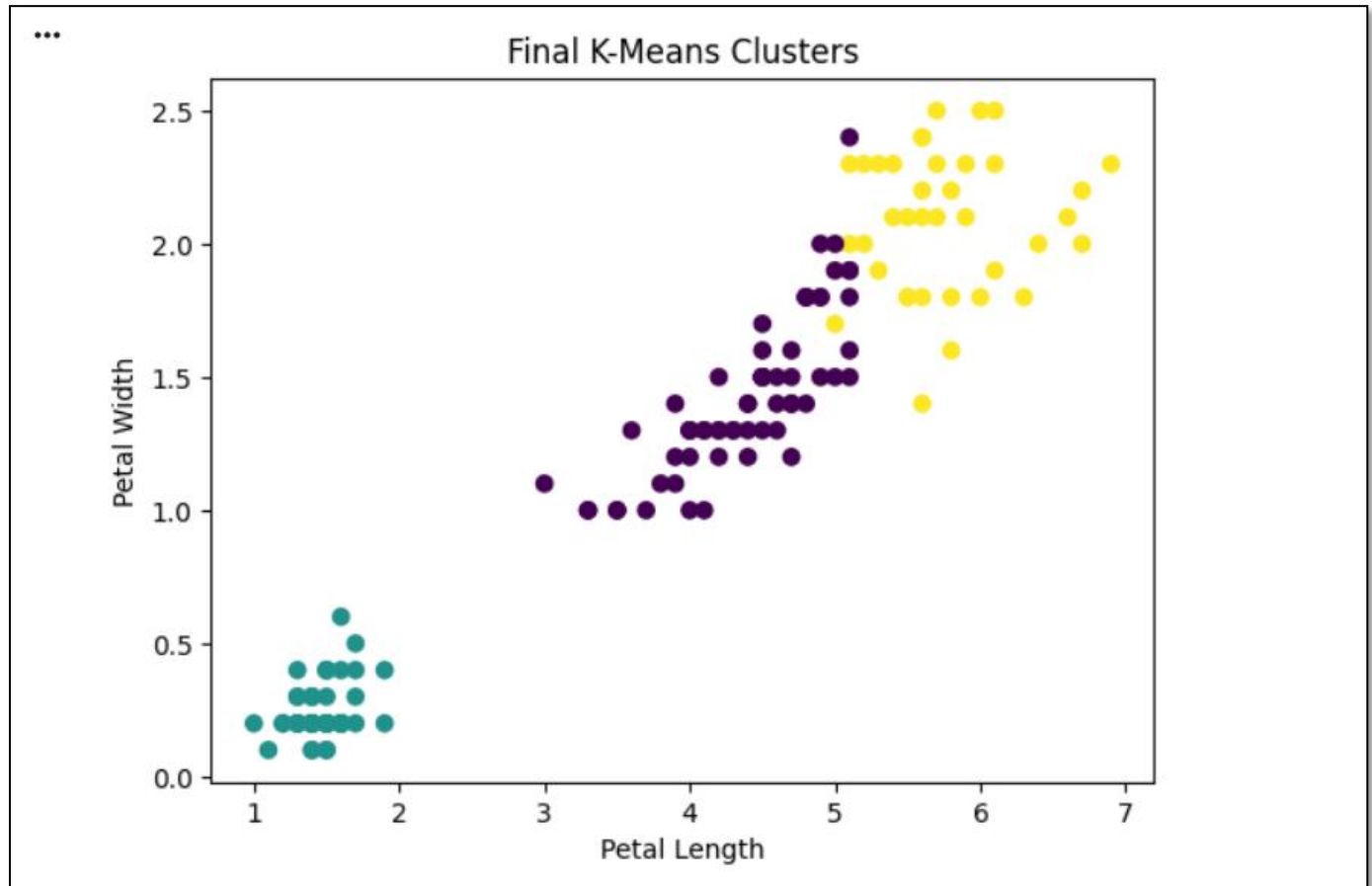
Step 15: Visualize Final Clusters using Petal Length and Petal Width

Code:

```
plt.scatter(
    X[:, 2],
    X[:, 3],
    c=final_labels
)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Final K-Means Clusters")
plt.show()
```

Observation:

- The final clusters were visualized using **Petal Length** on the X-axis and **Petal Width** on the Y-axis using the original feature values.
- The large centroid circles are not shown because this step plots only the observations, unlike Step 12 where the centroids were plotted separately.
- The graph shows clear separation of the low-petal-size cluster, while the other two clusters show some overlap.

Output:**Step 16: Calculate Silhouette Score****Code:**

```
silhouette=silhouette_score(  
    X_scaled,  
    final_labels  
)  
  
print(f"Silhouette Score: {silhouette:.4f}")
```

Output:

```
Silhouette Score: 0.4630
```

Observation:

- The Silhouette Score was calculated to evaluate the quality of the formed clusters based on their cohesion and separation.
- The obtained score is **0.4630**, indicating **moderate clustering quality**, with the clusters showing meaningful separation but some overlap.